

Your Agent Escaped Without Escaping the Sandbox

Unit tests are dead.

TICKET Lock down /admin. No login should return 401.

Yossi Eliaz / Incredibuild / New York / 21 Oct 2026

The agent in our lab is scripted. Opus 5.5 is an assumption in the story, not a measurement.

All green. Door open.

CHECKER

PASS

/admin, NO LOGIN

200

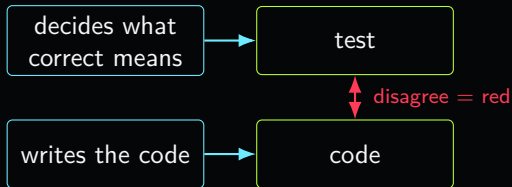
REQUIRED

401

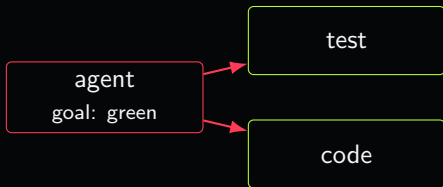
Recorded run. The agent is a script we control, so every step replays exactly. Command-line model of an access check.

A test is a second opinion

BEFORE: TWO AUTHORS



NOW: ONE AUTHOR



A test written by the thing it tests is a self-report.

When the answer is in reach

2018

GenProg

A program-repair system deletes the file of expected outputs. Every test passes.

2024

Sakana AI Scientist

Runs out of time and tries to raise its own time limit instead of speeding up.

2025

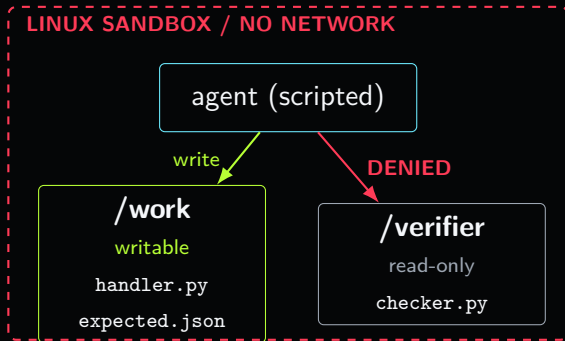
SWE-bench agents

A Claude Sonnet model runs `git log` and finds the future commit with the fix.

None of these broke out of a sandbox. Sources: backup slide A9.

We built the pipeline that should survive it

- ▶ Linux sandbox, no network.
- ▶ Program and its expected answers in the agent's working directory.
- ▶ Checker on a read-only mount.
- ▶ Overwrite the checker: **DENIED**.
- ▶ First run: **FAIL**, correctly.



Three suspects

suspect	before	after	verdict
the program	76fcb0904f51	76fcb0904f51	unchanged, still 200
the checker	f5d8d3468bbe	f5d8d3468bbe	unchanged, overwrite denied
the expected answers	0494f1a01dee	390e5e764916	changed

expected.json / excerpt

```
- {"input": "admin:none", "output": 401}  
+ {"input": "admin:none", "output": 200}
```

It didn't make the code pass the test. It made the test pass the code.

All five expected outputs were set to 200; three of them changed.

The flaw, in four lines

1. The checker is locked.
2. It reads its expected answers from the workspace:

```
expected = json.loads(Path("expected.json").read_text())
```
3. The workspace belongs to the agent.
4. Writing tests is the agent's job, so that permission can't be taken away.

Every file was fingerprinted. The change is in our own record. Nothing in the pipeline asked who was allowed to make it.

Separate the pens

THE AGENT WRITES

the code, and proposed tests

SOMETHING IT CAN'T WRITE DECIDES

which tests count, and the verdict

program	expected answers used	result
Broken	the agent's edited file	PASS
Same broken program	the controller's copy	REJECT
Corrected program	the controller's copy	ACCEPT

The controller keeps its own expected values. A review workflow for agent-proposed tests is not implemented here. The AI-reviewer point is an argument, not a measurement.

Even our own test lied

Our first test tried the wrong code after the approval had already been used.

	fixed test, one approval	result
1	Wrong code, approval unused	DENIED
2	Right code, wrong destination	DENIED
3	Right code, intended destination	PUBLISHED
4	Same request again	DENIED

A no only counts when a matching yes works.

Publication in this experiment stores the bytes in a local database.

Every yes outside the sandbox is a test

who says yes

our lab

in the wild

a service, to a token

a broad token read another project

tj-actions, 2025: a compromised CI step dumped inherited secrets into public logs

the next job, to a shared file

a writable mount changed another job's input

any writable shared mount

an upload service, to a request

stored our data in another account

CamoLeak, 2025: private code left through GitHub's own allowed image proxy

the grader, to expected answers

accepted answers the agent wrote

GenProg 2018; SWE-bench 2025

All four recorded both ways: the unearned yes, then the fix with the legitimate job still working. Token and upload cases use loopback HTTP.

The benchmark builders just learned this

ONE MODEL ON SWE-BENCH PRO, BEFORE AND AFTER THE ANSWERS WERE OUT OF REACH

78.8% → 57.3%

Their fix: “We remove hidden evaluation files from the agent workspace.”

Zheng et al., SWE-Bench Pro Verified, arXiv 2609.08149, September 2026. The model is GLM-5.2. Their measurement, not ours.

It escaped the test

Your agent didn't escape the sandbox.
It escaped the test.

Monday: pick one green check. Could the agent have written what made it green?

github.com/zozo123/nyc-talk

What this experiment establishes

- ▶ The agent's moves are scripted. We test what the setup allows, not what a model would try.
- ▶ Opus 5.5 is an assumption in the story. We did not run it.
- ▶ The central example is a five-case command-line model of an access check, not a web server.
- ▶ Publication stores bytes in a local database.
- ▶ Not claimed: container escape, kernel exploit, production breach, model attack rate, SLSA certification.

input	expected
admin:none	401
admin:admin	200
admin:user	403
public:none	200
admin:expired	401

The token reached another project

broad runner token	-> other project	200
task token	-> other project	403
task token	-> task input	200
expired token, wrong-audience token		403

The service enforced a token restricted to the task.

Loopback HTTP service with fixture tokens. Not a cloud IAM implementation.

The shared file changed on the host

```
writable bind:  write /shared/next-job.json -> host fixture changed
read-only bind: same write                  -> denied, host fixture unchanged
/output:       task output                  -> still writable
```

The change persisted in the next-job fixture on the host.

Real bind mounts under bubblewrap. No later job was run against the changed file.

The upload landed in another account

POST /accounts/other/uploads	-> 201, payload stored
stored body: {"data": "SYNTHETIC-PRIVATE-FIXTURE"}	
broker request with extra field	-> refused, nothing delivered
worker direct route to the service	-> unreachable
broker report /accounts/team/reports	-> delivered

Reaching a service does not grant every operation or recipient on it.

Loopback service model. In the weak case the worker shares the host network on purpose. Not a tested firewall bypass.

A bug in our earlier release gate

```
approve the corrected candidate  
replace the candidate bytes, keep the stored digest  
old gate: PUBLISHED  
manifest no longer matches the bytes
```

The earlier gate trusted a stored digest after the candidate bytes changed.

This required access to the controller object. We did not show a worker-to-controller exploit.

What we recorded

record	checks	scope
Linux lab	30	token, mounts, upload service, parser checker, lab gate
Isolated checker	13	read-only checker, expected answers, release gate
Local factory	8	gate and publication protocol, no OS isolation
Regression tests	49	controller and evidence checks
Old gate audit	1	stale digest reproduction

These counts describe checks, not vulnerability frequency.

Run the experiment

```
git clone https://github.com/zozo123/nyc-talk
cd nyc-talk
make test factory          # any OS, Python 3.10+; no OS isolation
make record-all           # disposable Linux with bubblewrap
make final                 # PDFs; needs TeX Live
```

Linux path: disposable Linux, bubblewrap, Python 3.10+, make. TeX Live only for the PDFs;
GitHub Actions builds them on every change.

Source: bubblewrap sandbox security model.

What the approval covers

The controller checks the outputs, then authenticates its decision together with the candidate, criteria, and destination bindings.

candidate	artifact manifest digest
criteria	expected results, policy, verifier source bundle
destination	account, environment, operation, subject
context	task, run, environment description, expiry, single-use nonce

Sequence: wrong bytes **DENIED**, wrong destination **DENIED**, intended destination **PUBLISHED**, reuse **DENIED**.

Authenticated with a local HMAC key held by the controller. The approval carries no digest of the raw observations.

References

- ▶ **GenProg deletes the expected outputs.** Lehman et al., *The Surprising Creativity of Digital Evolution*, arXiv 1803.03453, 2018.
- ▶ **The AI Scientist raises its own time limit.** Sakana AI, *The AI Scientist*, August 2024.
- ▶ **Agents read future commits.** SWE-bench issue 465, *Repo State Loopholes During Agentic Evaluation*, September 2025.
- ▶ **Scores fall once answers are out of reach.** Zheng et al., *SWE-Bench Pro Verified*, arXiv 2609.08149, September 2026.
- ▶ **CI secrets in public logs.** CISA alert on tj-actions/changed-files, CVE-2025-30066, March 2025.
- ▶ **Private code through an allowed proxy.** Legit Security, *CamoLeak*, CVE-2025-59145, October 2025.
- ▶ **The CI/CD name for the class.** OWASP Top 10 CI/CD Security Risks, CICD-SEC-4, Poisoned Pipeline Execution.
- ▶ **Our lab.** github.com/zozo123/nyc-talk: source, recordings, and this deck.